context of Parca, the team decided to develop FrostDB (https://github.com/polarsignals/frostdb), a dedicated columnar database for profiles, written in Go. You can embed FrostDB, and it features semistructured schemas (where some or all of the schema fields and or types are unknown or undefined) using Apache Parquet for storage and Apache Arrow for querying.

- *To minimize the storage footprint for the timestamps, continuous profiling backends typically use XOR compression.* Think of it as a delta-of-delta encoding for timestamps. This technique was first described by a team of Facebook engineers in "Gorilla: A Fast, Scalable, In-Memory Time Series Database" by Tuomas Pelkonen et al. (2015, Proceedings of the VLDB Endowment; http://www.vldb.org/pvldb/vol8/p1816-teller.pdf). If you want to learn more about compression in this domain, check out "Time-Series Compression Algorithms, Explained" by Joshua Lockerman and Ajay Kulkarni (https://www.timescale.com/blog/time-series-compression-algorithms-explained/).

Once stored efficiently, you will want to query profiles. So what's the challenge here?

9.3.3 Querying profiles

Ideally, we would be able to support expressive queries for profiles. Currently, mostly simple, label-based query languages, such as in Parca (https://www.parca.dev/docs/querying-parca) and Pyroscope's FlameQL (https://pyroscope.io/docs/flameql/), exist. This is not a bad thing, per se, since the widely used and popular query language for Prometheus, PromQL, is also a label-based query language. The question here, however, is if and when an (open) standard will emerge as well as whether it will be an SQL-related or a DSL-related format.

9.3.4 Correlation

One of the key challenges, as well as promises, in the developer observability space is correlation. Naturally, one would expect to jump from other signal types, such as a distributed trace or metric, to profiles (be that label-based or via frontend support, as with a Grafana plug-in). This is an area of active research, and we can expect more to happen here. We will cover the topic in greater detail in chapter 11.

9.3.5 Standards

We already discussed standards a couple of times; the gist is that you want to make sure the tools you are using to achieve developer observability are built atop and use open standards, including OpenTelemetry, for signals on the wire, as well as up-and-coming standards, like `pprof`, for representing profiles on the wire.

9.3.6 Using tooling in production

You can use continuous profiling in development environments as well as in production environments. There, you will need to come up with a strategy concerning the